QCon
全球软件开发大会

InfoQ 技术大会

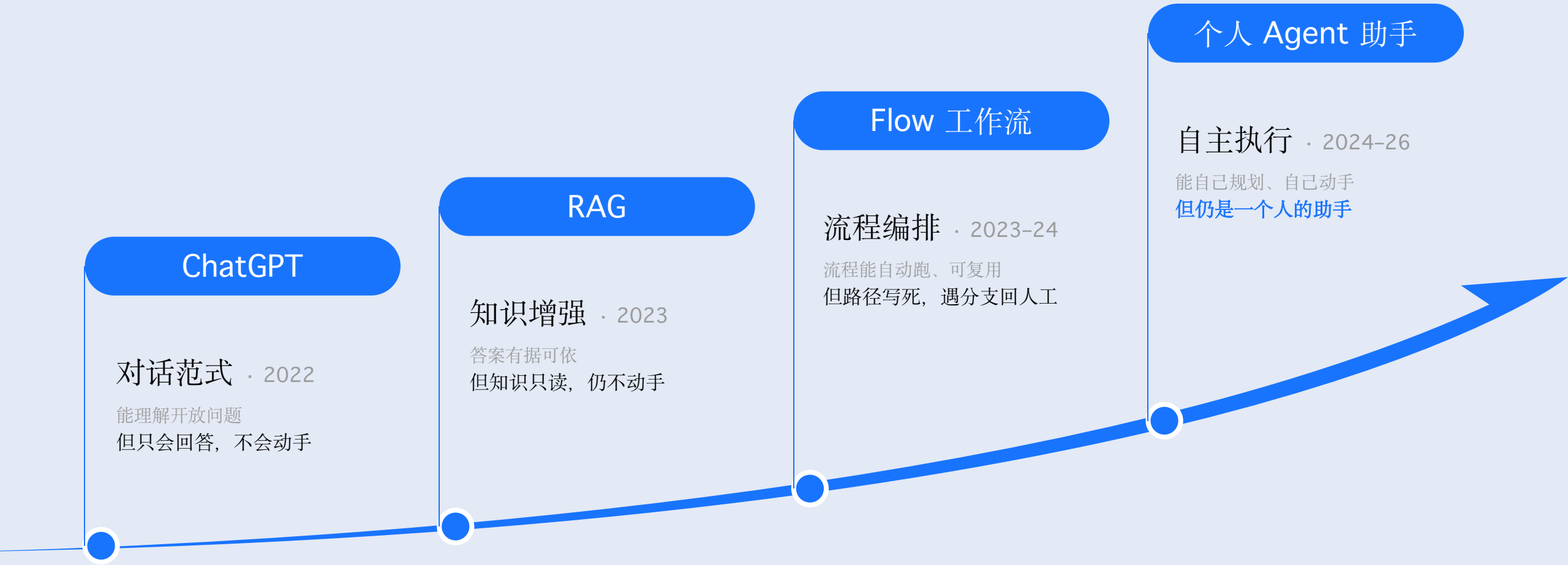
从“会用 Agent”到“组织级生产力”
网易帝王蟹多 Agent 设计与实践

裴明明

网易智企 AI 平台技术负责人



AI Agent 的四次跃迁：从回答问题，到完成任务



今天的 Agent 助手能做什么

复杂任务，要多个不同能力的 Agent 协作；每个 Agent 能力内聚
不是能力不够，是一个 Agent 做不了所有的事情



理解目标

把用户的一句输入，变成可执行的目标与验收标准
复杂任务容易理解偏差，理解效果完全依赖模型



调用工具

搜索、读写文件、执行命令、访问外部系统
工具自选、多步自串联；工具一多，执行效果便不可控



记忆上下文

记住历史与偏好，跨轮次保持连贯
但它只有一份记忆，全部注入即会膨胀，压缩难度同样很大



自我反思

会复盘、会重试、会调整路径再试一次
错误自检、自行修正，无需人工盯守

单 Agent 的五个限制

五条指向同一个原因：Agent 只有一个上下文——一份注意力、一套能力、一条时间线、一个视角。

01	一次请求的平均固定上下文	16,667 tokens
02	工具池扩大后的选对率	13.6%
03	可并行的独立时间线	1 条
04	重启后可自动续接的节点	0 个
05	模型自查不出的错误	64.5%

多 Agent 的价值与必然性：能提升人效、能可靠产出

背景

行业方向已经收敛：按专业拆分角色、把协作固化成流程，或把单 Agent 做到极致。共识只有一句——难点不在模型，而在协作与治理；从能力到产出，还差一层秩序。

实现

角色分工定边界，统一编排定流程，状态记录定接续，独立验收定可信。四件缺一件，就退化成各干各的模型，人越多反而越乱；补齐四件，单点能力才能变成可复制的生产力。



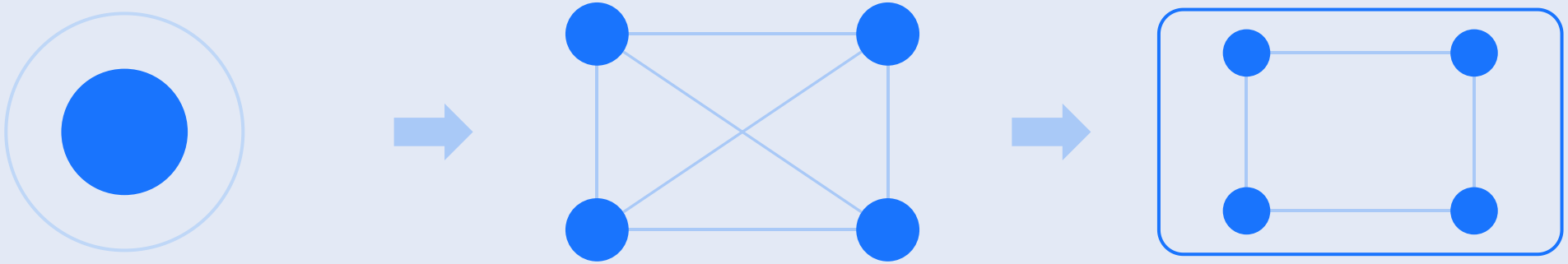
价值

要成为生产力，必须能提升人效、能可靠产出。单 Agent 是基础能力，多 Agent 才是生产力——它本身就是一支组织，效率才做得出来。

收益

每件事有唯一负责人，每一步都可追踪，每份产出都可核对，每次运行都能沉淀。Agent 是组织里的数字成员：人效上去了、产出稳得住，经验从个人手里变成组织资产。

多 Agent 与组织：角色即员工，关系即组织



岗位

一个角色被赋予职责与权限，
它就是组织里的一名员工。

关系

员工一多，就长出协作方式
与考核机制——成果验收，
是其中最硬的那项指标。

组织

这些关系连成一张网；
这张网反向定义了组织。

多 Agent 不是“多了几个 Agent”——而是一支有职责、有协作、有考核的队伍。

多 Agent 的实现：四件事，缺一不可

能力叠加只是起点；Agent 成为组织的一员，取决于这四件事。

四个面	为什么看重它	难在哪
角色与边界	**拆分是为了聚焦——边界划定后，prompt 专注、上下文互不干扰**	通用模型不会自带专业——prompt、Skill、MCP 一件件装配，才成为专家
编排与依赖	**拆分之后仍需合成一次交付——先后与依赖必须落成结构**	失败大多出在交接处——依赖不落结构，就会顺着往下传
状态与恢复	**链路一长、跨了会话，须有统一状态记录——承接、流转、推进均依赖它**	状态要准确、要能校验、要能持久化——多份“像真源”的必然打架
校验与质量	**拆分后多主体协作，合并质量无人担保——须逐段校验**	错误会跨 Agent 向下传播——校验不独立，等同自检自证

不是模型能力不足，是这四件事无法回避。

帝王蟹多 Agent 总架构：平台管秩序

ClawHive / OpenClaw Runtime

控制面 · 决定怎么做

入口 / Session
App · WebChat · IM

↓

路由与分流
keywor
d · semantic · memory

↓

主控 main
理解 · 规划 · 调度 · 终止

↓

Workflow Runtime
Graph · Gate · Scheduler

↓

Finalizer · 最终交付

执行面 · 动态装配的 Agent Worker Pool

不是固定角色清单：按任务需要装配专家、执行者和验证者

Agent Worker Pool
角色由规划定义，能力由配置注入

每个 Worker 的边界

- 职责 Task owner
- 上下文 Session / memory
- 能力 Skill / Tool / MCP
- 约束 Policy / Output

Task / Stage / Dependency
主控只派发可运行节点；依赖未满足，节点不启动

→

TaskResult + 摘要 + 产物引用 · 不回读草稿，不复制正文

治理侧栏 · 横切所有层

- 上下文边界 / 共享摘要与引用，隔离局部推理
- 执行门禁 / 计划确认、owner、依赖、权限
- 驱动与执行 / 渐进式推进、交互确认、状态与产物检测
- 证据与观测 / 进度、日志、产物、成本、审计

四平面：平台管编排秩序，Harness 管单 Agent 秩序

平台管编排秩序：怎么拆、怎么推、何时停，外加安全与成本；内容对不对，不归它判断

控制面

执行者管自己那一个任务：内容与专业判断自己说了算，边界外不插手

执行面

平台统一接能力：Skill、工具、MCP、知识库走同一套接口，谁也不能各接一套

能力面

Harness 管单 Agent 秩序：上下文、记忆、循环与产物；跨 Agent 的协作不归它

治理面

帝王蟹的一次任务链路：从一句话，到可验证的交付

1 理解目标
Intent / Constraint
用户诉求 · 成功标准

2 协作设计
Role / Boundary
分工 · 边界 · 策略

3 任务规划
Graph / Dependency
任务图 · owner · 依赖

4 有界执行
Context / Policy
上下文 · 能力 · 权限

5 校验归因
Evidence / Review
证据 · 复核 · 归因

6 收敛交付
Deliver / Learn
结果 · 状态 · 经验

前向：能力如何被组织

把复杂目标压缩为可协作的任务结构

边界清晰 → 上下文可传 → 依赖可调度

任务图承接分工；唯一 owner 负责收敛；阶段门禁阻断越级执行。

反向：质量如何被证明

把每次协作沉淀为可验收的状态转换

结果有证据 → 失败可归因 → 状态可恢复

TaskResult 经校验才推进。

异常不等于重试：超时 / 缺结果 / 冲突 → 归因 → 注入现状和目标继续驱动 → 恢复 / 终止

任务前进由依赖决定；任务完成由证据决定。

一次任务的价值，不止是完成，更是留下可复用的协作经验。

角色边界：配什么能力，成什么角色

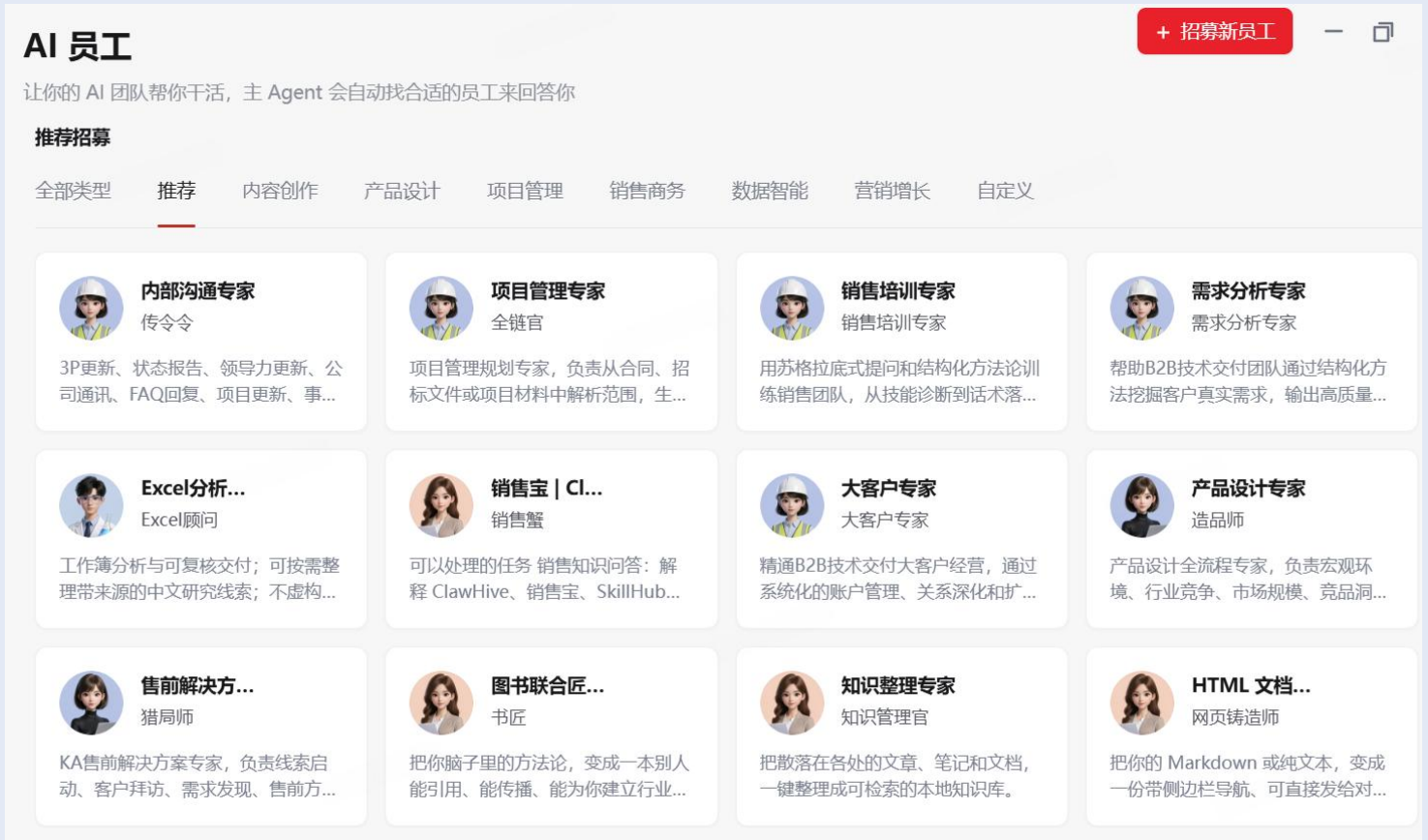
同一个Agent，配上不同的上下文、工具、权限，就是不同的角色。

主控 Agent

只做四件事：理解目标、规划、调度、风险控制与最终交付——它装的是专业流程，也读得懂每个领域 Agent 的能力边界，所以派得准

执行 Agent

以唯一任务所有者的身份，专注做本领域的专业工作——通用模型只是底座，身份与规则、工具与技能由 plugins 与 MCP 两类协议装载，记忆只服务这一件事，越用越专业



一个角色，一套专属环境

各自的上下文

只注入完成本职任务所需的信息，历史不全量注入模型——项目文件、上游产物与规则按需提供

各自的记忆

只装载职责所需记忆，按内容类型与标签召回，不共用一份持续膨胀的记忆——其他角色不可见



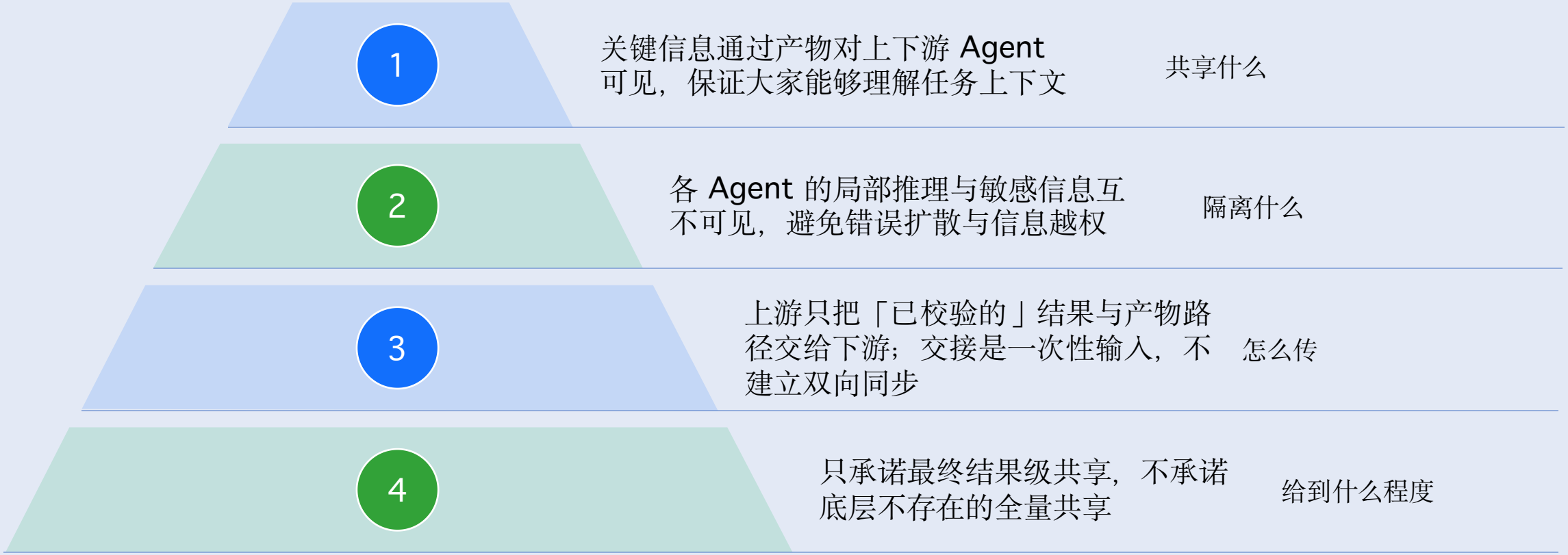
各自的工具与技能

按职责接入，以对口为先而非越多越好——仅挂载角色所需工具

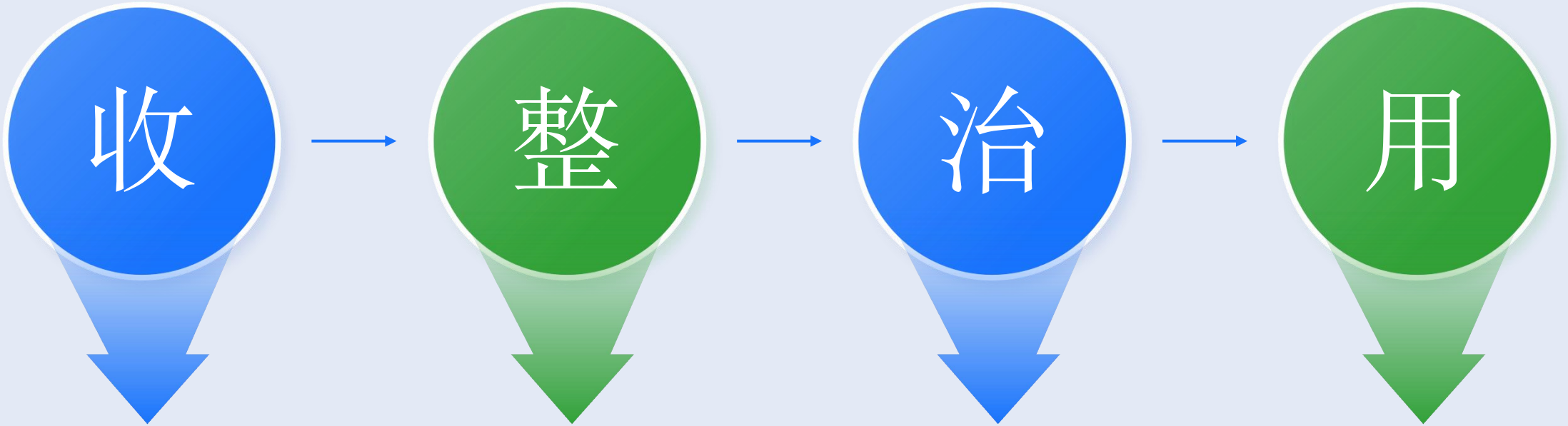
各自的工作区

独立的 workspace、会话与产物目录，互不污染、互不可见——身份与规则随角色走

上下文怎么共享，怎么隔离



记忆和知识管理：从原始记录，到可用知识



Session 存成本地文件：
对话、工具调用等记录，
按角色隔离

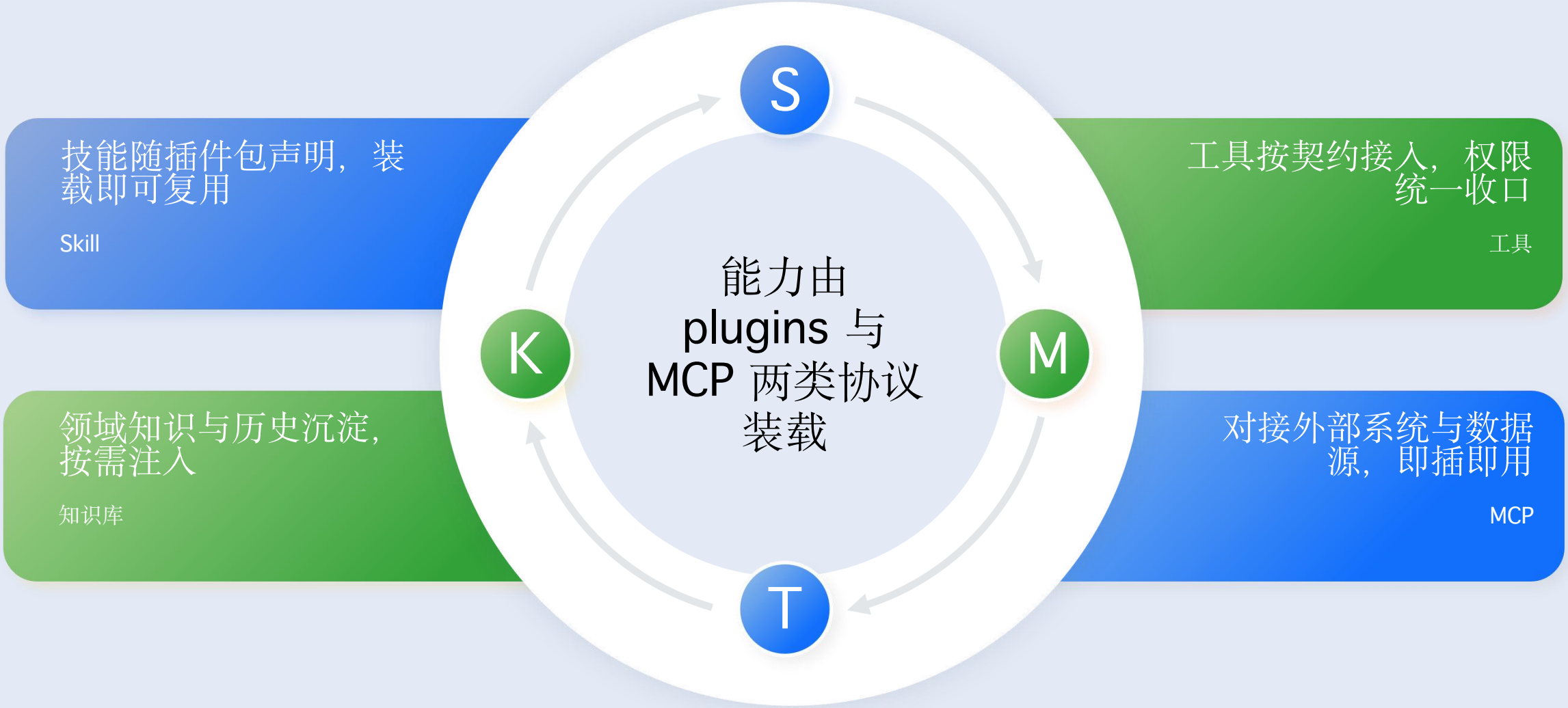
无序的对话，整理成
结构化的记忆数据，
即会话记忆与领域记忆

提炼记忆，识别出
高频主题、整理成文档；
用户审核后合并更新入库

知识按需召回、直接用于
任务；新的产出又回流，
成为下一轮的输入

从原始记录到可用知识，中间每一步都要管。

能力接入：四类标准接口


技能随插件包声明，装载即可复用
Skill
工具按契约接入，权限统一收口
工具
领域知识与历史沉淀，按需注入
知识库
对接外部系统与数据源，即插即用
MCP
S
K
M
T
能力由 plugins 与 MCP 两类协议装载

意图识别：从用户输入到「任务由谁来处理」

识别不是让大模型直接判断：以任务比对现有角色池，规则、向量、小模型逐层出分，不足才交由大模型兜底

1 先判任务

Intent / Constraint

目标 · 动作 · 交付物

命中任务动词或能力词才成立；闲聊、概念提问不进入流程

2 逐层识别

Cascade Routing

规则+向量→小模型→LLM

规则+向量出分，小模型给置信度；0.95 / 0.85 / 0.72 分档，不足下沉

3 读出规模

Activation Gate

角色数 · 能力面 · 协作意图

角色池不同答案不同；≥2 个角色且能力面不同，才可拆分

4 只出候选

Candidate ≠ Dispatch

角色存在 · 置信度 · 路由一致

四道闸：角色启用、置信度达标、路由一致、不落兜底

拆分判定：为什么值得多 Agent

不是命中多个词，而是能力面真的分开：判断、信息、权限不同

能力隔离	上下文隔离	协作意图
不同专业判断	不同信息与权限	需要并行或分阶段

反向判据：写操作重、强依赖、共享状态 → 倾向单 Agent

识别与派发：两件事不能混

识别只缩小选择空间、不对动作负责；派发由状态机按依赖决定

输入 → 候选 → 状态机 → 派发 / 不派发

无法判定不是失败，而是升级信号：本地判不出则向上移交，不硬判

判定输出：chat / single_agent / multi_agent ·　识别给候选，状态机给动作

任务规划：从信息输入，到一张 DAG

1 信息输入 · 输入什么

用户目标、上下文与已有产物汇成一次输入——规划以信息为前提

2 大模型规划 · 谁在规划

由大模型基于信息输入产出步骤与分工——这一步只出计划，不落执行动作



3 DAG 编排 · 怎么成图

计划落成有向无环图：节点是任务、边是依赖——顺序由图定，不由对话定

4 主控执行 · 谁在调度

主控按图调度与收口，不直接执行——调度对象是任务图，而非一段对话

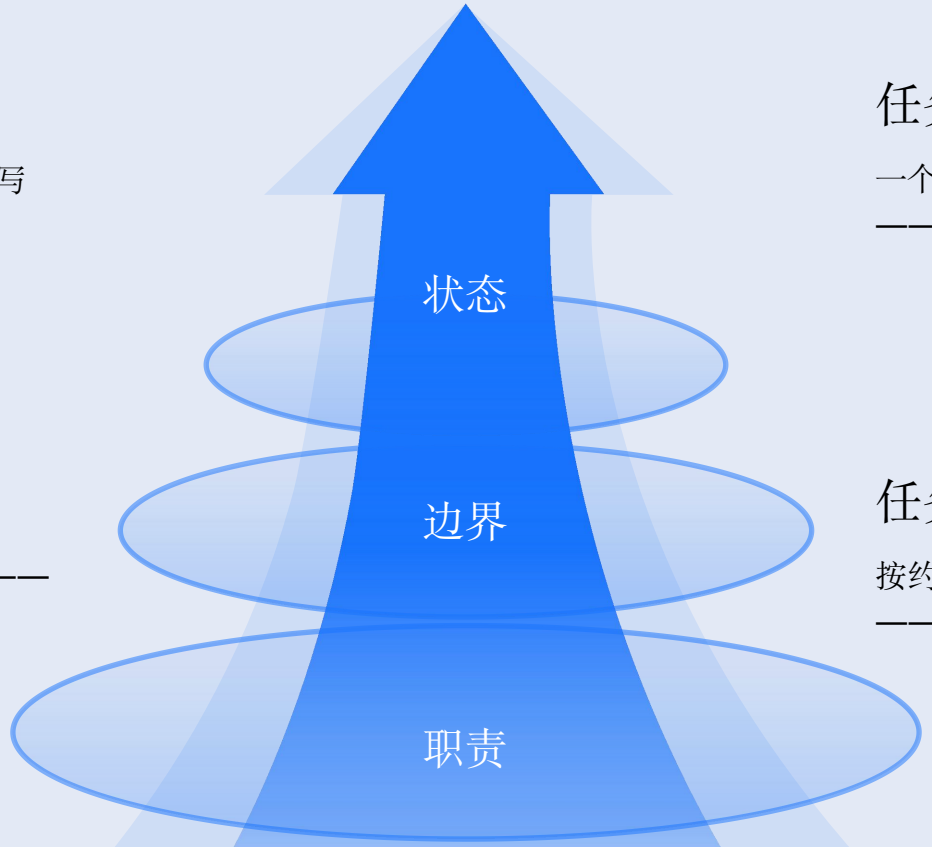
任务管理：职责、角色、边界、输入与输出

角色定义 · 谁来做

每个节点绑定唯一角色并写明 agent id——不写“任意角色可承接”

任务描述 · 做什么

一个节点一件事，粒度小到可验收——不写含糊描述



任务输入 · 从哪来

上游产物按依赖给到，只给最小充分的上下文——不复制正文

任务输出 · 交什么

按约定格式产出并登记结果清单——产物过检，方算交付

Agent 间交互：对齐 A2A 的五个构件

Agent 之间是结构化交接：能力如何声明、任务如何跟踪、上下文如何传递、产物如何交付、用户交互。

环节	A2A 构件	协议定义	我们的实践
能力发现	Agent Card	能力与技能的可发现声明	每个角色一份能力声明，写在角色定义里
任务交接	Task	带标识与状态的任务对象	nodeId 与状态文件，按图推进
上下文传递	Message / Part	结构化消息，内容单元化	只传摘要与产物路径，不复制正文
产物交付	Artifact	与任务绑定的产物	交付物和状态清单完整才行
用户交互	input-required	需要外部输入的标准状态	生命周期里的正常一站，需澄清或授权时停下

跨组织、跨框架的互通可以直接采用 A2A；同一支团队内部，也可以用插件自行实现这几个构件——结构对齐，方式可选。

多Agent任务流程管理

流程异常没有一份有限清单：模型输出不守协议、业务边界不明确、用户意图不确定。

① 输出格式漂移

status 写成 done、完成、finished；协议里要的是 completed。
产物写成 review.md；dependsOn 写成一句「依赖 A 的调研结果」。
同一个字段，换个模型、换种语言，可能又换一个词，也可能干脆不写。

枚举当前模型的输出变体，再次执行仍可能不同。

② 业务边界不明确

针对不同的业务场景，比如「整理合并表格」，模型去做了。
但是主Agent校验不通过，流程上如何进一步推进执行Agent去修正和处理。
或者业务结果路径跟预期不符的时候如何继续处理。

代码没有预设的业务场景，结果不符预期时如何处理?

③ 意图不可判定

它问「这份合同按哪个版本拆」，用户回「取消当前任务，改成生成一份脚本」。
这句可能是真取消，可能是先否掉上一步再给新指示，也可能是回答「换新版本」。
三种读法都成立；代码只能按字面读，按字面办，任务就没了。

把说法逐条枚举，用户能说出多少种?

流程异常没有清单，硬编码无法穷举——我们换一种流程推进方式。

渐进式推进：主控通过目标对齐的方式推进流程

LOOP：Agent结束不等于任务完成

专家回传只是一次运行结束，不等于任务完成。主控先核对产物与结果清单——自述“已完成”不构成完成依据。



上下文注入：每轮把状态给全

当前状态、待办任务、预期状态、预期产物，每轮一次给全。提示词与状态机同源，重新解析，不沿用聊天记录续接。

流程推进：只认机器可读输出

解析到约定的结构化输出，状态才发生转移；未解析到则停留在原态，下一轮明确指出缺失项。表述再肯定，也不改变状态。

任务收口：产物验证通过才算走完

上游产物原样交给下游，主控不生成中间总结；仍有节点在执行，不做阶段总结、不推进下游——每轮都以任务状态和目标产物收尾。

状态机：一次任务要走过的状态

拆解与派发之后，首先要回答：当前执行到哪一步。



流程状态由主控驱动；任务状态由专家驱动（pending / running / completed / failed）

中断与恢复：状态在进程之外，恢复才有依据

一次任务要跑十几个节点，进程重启、会话丢失是常态；能否续上，取决于状态是否已经离开进程。

状态存在哪　提交以 workflow.json 为准，索引可重建

.workflows/	任务状态根目录
index.json	Workflow 摘要索引，可重建的派生缓存
<workflowId>/	一次任务一个目录
workflow.json	阶段、依赖、事件、已验证任务、提交版本
tasks/<taskId>.json	状态、执行会话、结果候选

写盘　单文件临时写入后整份替换，不留半写

提交　任务文件是候选；workflow.json 落盘即提交，索引最后更新

断了怎么接　启动、判定、推进，各管一件事

01　启动

先取驱动租约，再做检查与推进；未持租约不自动派发，只认明确继续

02　判定

有活跃执行→　等结果，不重发

无活跃执行→　先取该次结果：有效即提交，无效才收回重做

查不到　→　暂缓不猜；超时且收到明确终止，按孤儿任务回收

03　推进

只认已提交的节点，未提交的不进入下游

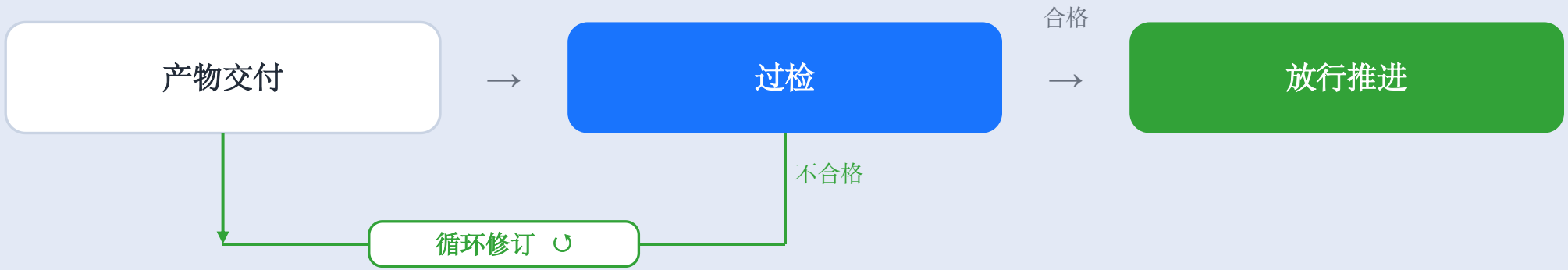
对账按执行轮次一次；投递带幂等键，事件按事件号去重

注入　**恢复的关键**：与每轮推进同一套，即当前状态、待办、预期产物；只重派未提交节点

状态可读，恢复才有起点。

检验与质量：各判其责，循环修订

质量不足的产物，会沿依赖链逐级放大。



检验 · 存在性 流程判

状态是否更新 · 清单是否齐全 · 字段是否合规

质量 · 合规性 专家判

内容是否合规 · 是否达到约定的验收标准

两层各判其责，逐层放行——判据都是机器事实：文件、清单、状态，可核对、可追溯。
未通过则回到修订、重新送检，循环往复，直至通过。

完成的定义：以记录与产物为证

判据只认留下的证据：一份记录，一批产物。

01 **表述不成立完成** 对话回复、口头说明，均不构成完成的事实。

02 **记录须可校验** 字段固定、值域封闭；缺项即判不了，不是判失败。

03 **产物须独立成件** 单独成文件，且列入记录声明的清单；同名、空文件不计。

判据形态

规则固定

只看形态，与任务内容无关。

规则随任务约定

写在交付标准里，由人先定。

表述不成立，记录与产物才成立。

结构校验：一份清单，十三项硬门禁

路径、归属、摘要、产物类型、下游节点——先合法，再判产物。

执行者提交的结果清单

```
---
nodeId: A
status: finished
summary: Docker review finished.
artifacts: [artifacts/A-review.md]
---
```

核对这份清单：两处不通过

① **status 写成 finished**

合法写法只有 pending / running / completed / failed

写法不在枚举内 → 节点判 failed，不进入下游

② **artifacts/A-review.md**

清单已声明，任务目录内无此文件

已声明但不存在 → 停留于「待用户确认」

清单与路径 · 4 项

- 落 artifacts/{nodeId}-result.md
- nodeId 与路径一致
- 节点在计划图内
- front matter 齐全

摘要与产物 · 4 项

- 摘要非空
- 产物非空
- 不是系统文件
- 声明即须存在

推进之前 · 5 项

- 下游节点存在
- 确有依赖关系
- 状态写法可识别
- 路径安全
- 不越出任务目录

形式合法，方进入内容评价。

内容校验：执行 Agent 产出物的验收闭环

以灵感生图专家为例：产物是图片。出图不等于交付，须经视觉复核逐项通过，方可放行。

验收闭环 循环至通过，或达上限为止

01 生成

内置 image_generate / 依需求描述与画幅要求 / 生成图片文件

02 视觉复核

Agent 主动调用 image 工具 / 成品图交视觉模型 / 按清单逐项检查，返回评审意见

03 判定

通过 → 放行 / 未通过 → 重生成 / 达上限 → 放弃

04 放行

复核通过才调用 present_files / 给出交付说明 / 模型 · 轮次 · 已知限制

复核清单 视觉模型逐项检查，逐条给结论

维度	检查内容	不通过判定
文字	逐字转写图中全部中文，核对错别字、乱码、缺字	任一字符错误
人物	判断形象是否专业，排查贴纸、畸形与异常元素	出现上述任一项
版面	核对排版层级、主体位置与文字区清晰度	层级不可读或文字区被遮挡
一致性	比对产品外观、服装、背景与需求描述	任一项与需求不符

意见 须指明位置与改法；不可执行的评语不能作为重生成输入。

终止 达上限仍不达标：停止重试，说明限制，失败品不作交付。

复核通过，方进入交付。

从“跑起来”，到“跑得稳”

稳定性的来源不是更聪明的模型，
而是更清楚的边界、更完整的上下文、更硬的验收，和更快的反馈

1 可观测 · 看得见

进度、日志、产物与成本，每一步都留下可查的证据——先看得见状态，才谈得上自主。

2 知识管理 · 记得住

把一轮任务的经验升成共享知识——不随会话丢掉，记得住，下一轮不必重来。

3 自我学习 · 可沉淀

能够不断从可观测系统中采集信息，分析可优化提升的点，实现多Agent流程、Skill、工具、角色配置等持续迭代进化

四件事让它完成任务；这三件事，让它每一轮都更好——看得见、记得住、可沉淀。

可观测：判断的前提是状态可见

Agent 的每一个判断，都建立在对「现在是什么状态」的认知上——看不见，无从判断。

任务状态与阶段

每个节点处于什么状态、受阻于哪个依赖、下一阶段何时可开——可直接判断是否需催办

推进到哪

模型与工具调用

调用对象、次数与重试轮次全程留痕——出问题时可回溯到具体调用

做了什么

产物与凭证

产物文件与凭证是否齐全、是否列入结果清单——交付凭据可即时核对

交付什么

耗时与成本

每个角色的成本——按角色归因，才能定位空转与浪费环节

耗费多少

组织知识管理：多个通道汇聚知识库

可汇入

多个通道汇入：经验、文档、录音等，都能进组织

可治理

过审核、带权限、有版本；写入须经授权与审核

可复用

按标签共享到全组织，任何角色都取到同一份结论

1

多通道汇入

Agent 记忆、资料上传、录音沉淀等——都能进组织知识库

2

审核与权限

未经治理不进库；未授权的资料，也不进检索与上下文

3

全组织复用

一次沉淀、所有角色都能用；同类问题无需重复

Agent 经验 · 文档资料 · 业务数据 · 会议录音 → 统一入知识库，全组织按标签取用

自我进化：EERO 闭环，让每一次运行都成为下一轮的输入

Execute → Evaluate → Reflect → Optimize



- 执行
  - 每次执行都留下完整轨迹——调用、产物、结论，都是学习的原料
- 评估
  - 用评测集检查结果与过程，定位问题出在哪个环节、哪一步
- 反思
  - 把成功与失败都拆成可复用的经验，写清做法、参数与判据
- 优化
  - 经验回写 Skill、配置与评测集，后续轮次直接受益，越用越准

组织级生产力的提升

01 产出

人效提升

3–5×

同样团队能接的复杂任务数，人只做验收与调优

02 质量

任务成功率

99.5%

通过结构校验与内容验收，无需返工

03 代价

人工介入率

11.5%

需要人确认或补输入的任务占比

交付提速

1.6–2.2×

周期缩短至 1/1.6–1/2.2

工具与 Skill 调用准确率

94.2%

参数与目标工具一致；失败多在规则没记全

五项指标同一口径、逐周更新：人效与周期看产出，成功率与准确率看质量，介入率看代价。

组织级生产力，是产出、质量、代价三项同时改善。

多 Agent 的长期价值：不只是完成更多任务

短期可调度，中期可进化，长期是组织级生产力。

可调度、可治理

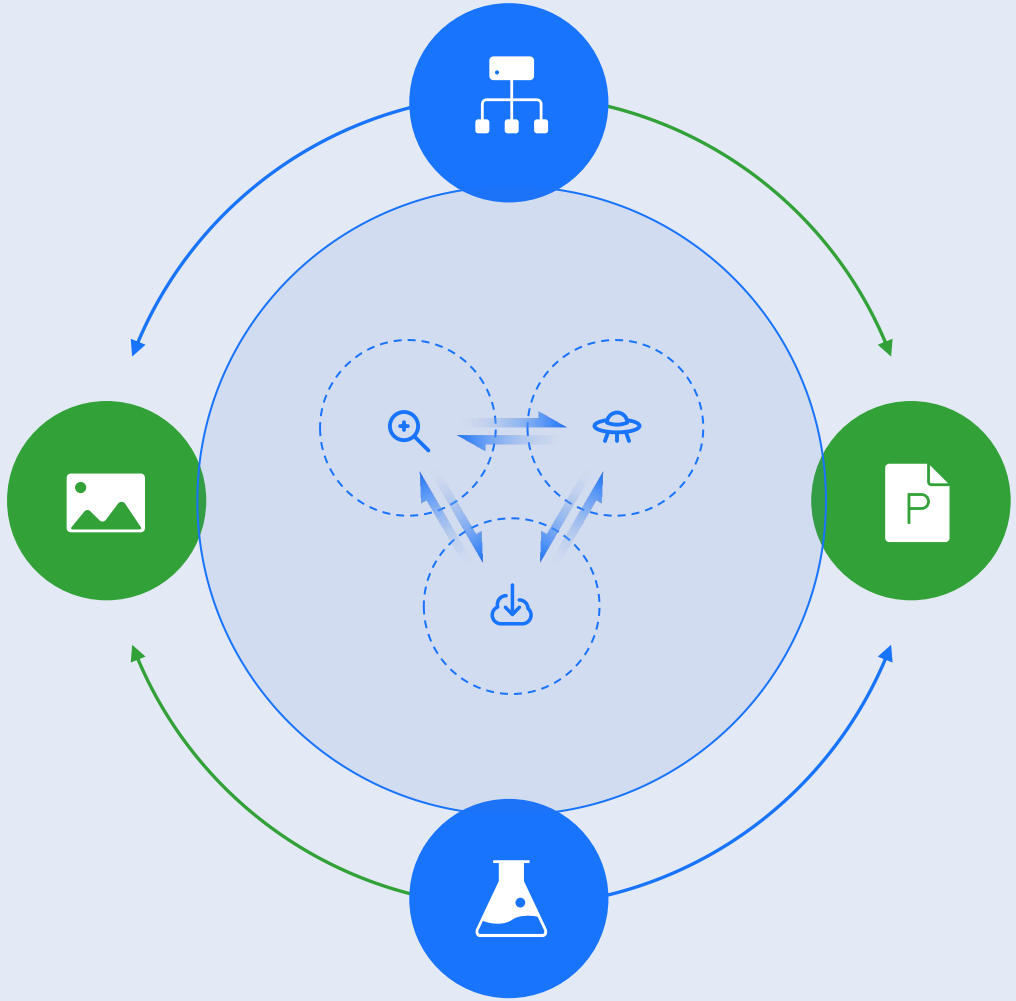
短期要让每次协作有秩序——边界清楚、过程可见、结果可验

可持续进化

中期要能把每次执行沉淀下来——成功与失败都变成组织资产

组织级生产力

长期是组织形态变革，充分释放组织生产力



AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

从“会用 Agent”到
“组织级生产力”

QCon 上海 2026 · 裴明明

